```python
import pandas as pd

df = pd.read_csv("Project2_data_2025.csv")
print(df.shape)
df = df.drop(columns=["Unnamed: 0"])
df = df.dropna()
print(df.shape)
print(df.head())
```

```
(1048570, 14)
(1048526, 13)
   ORIG_AMT  ORIG_TRM  NUM_BO  DTI PROP_TYP  reperformance_flag
Loan.Age  \
0        95       360       2   30       SF                   0
10
1        95       360       1   55       SF                   0
44
2       242       360       1   50       SF                   0
11
3        92       360       2   50       SF                   0
3
4       155       360       2   30       SF                   0
22

   VinYr  FicoBkt    LAST_VAL  MLTV  Default STATE
0   2000      780  105.298846    90       -1    IL
1   2000      740  214.892455    45       -1    IL
2   2000      700  331.233585    75       -1    CA
3   2000      780  101.799593    95       -1    OH
4   2000      780  268.251889    60       -1    MN
```

```python
import numpy as np

from sklearn.model_selection import GridSearchCV, RandomizedSearchCV,
KFold
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
from sklearn.neural_network import MLPClassifier

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
from scikeras.wrappers import KerasClassifier
```

```
train_df = df[df['VinYr'] == 2000].drop(columns=['STATE', 'VinYr'])
test_df  = df[df['VinYr'] == 2001].drop(columns=['STATE', 'VinYr'])

# 3. Original targets
y_train_orig = train_df['Default']   # values: 0 = default (positive),
-1 = non-default (negative)
y_test_orig  = test_df['Default']
X_train_full = train_df.drop(columns=['Default'])
X_test       = test_df.drop(columns=['Default'])

# 4. Recode to {0,1} for classifiers: 1=default, 0=non-default
y_train_full = (y_train_orig == 0).astype(int)
y_test       = (y_test_orig  == 0).astype(int)

numeric_cols =
X_train_full.select_dtypes(include=['number']).columns.tolist()
cat_cols     =
X_train_full.select_dtypes(include=['object','category']).columns.toli
st()
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numeric_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore',
sparse_output=False), cat_cols),
])

idx40k      = X_train_full.sample(n=40000, random_state=42).index
X_sub       = X_train_full.loc[idx40k]
y_sub       = y_train_full.loc[idx40k]        # {0,1}
y_sub_orig  = y_train_orig.loc[idx40k]        # {0,-1}

X_tune, X_val, y_tune, y_val, y_tune_orig, y_val_orig =
train_test_split(
    X_sub, y_sub, y_sub_orig,
    test_size=0.2, random_state=42, stratify=y_sub
)

# 6. Calculate scale_pos_weight
# 6a. count how many 0s and 1s in tuning data
n_negative = (y_tune == 0).sum()
n_positive = (y_tune == 1).sum()
scale_pos_weight = n_negative / n_positive

print(f"scale_pos_weight = {scale_pos_weight:.2f}")
```

```
scale_pos_weight = 38.65
```

```
# 11. Evaluation on test set with original labels
def evaluate(model, X, y_orig, is_rnn=False):
```

```
    if is_rnn:
        X_proc = preprocessor.transform(X).reshape(-
1,1,preprocessor.transform(X).shape[1])
        y_bin_pred = (model.predict(X_proc).ravel()>0.5).astype(int)
    else:
        y_bin_pred = model.predict(X)
    # map back to original labels
    y_pred = np.where(y_bin_pred==1, 0, -1)
    cm = confusion_matrix(y_orig, y_pred, labels=[-1,0])
    tn, fp, fn, tp = cm.ravel()
    print(f"\n=== {model.steps[-1][0]} ===")
    print("Confusion matrix (true [-1,0] vs pred [-1,0]):\n", cm)
    print("Accuracy:      ", (tp+tn)/(tp+tn+fp+fn))
    print("Precision(0):  ", tp/(tp+fp) if tp+fp>0 else 0)
    print("Recall(0):     ", tp/(tp+fn) if tp+fn>0 else 0)
    print("Specificity:   ", tn/(tn+fp) if tn+fp>0 else 0)
    print("F1(0):         ", 2*tp/(2*tp+fp+fn) if 2*tp+fp+fn>0 else 0)


# 7. Helper to get F1 on validation
def f1_val(pipeline):
    y_bin_pred = pipeline.predict(X_val)

    return f1_score(y_val, y_bin_pred, pos_label=1)


# 8a. XGBoost manual grid search
pipe_xgb = Pipeline([
    ('pre', preprocessor),
    ('clf', XGBClassifier(
        eval_metric='logloss', random_state=42,
scale_pos_weight=scale_pos_weight
    ))
])

best_score, best_params_xgb = -1, {}
results_xgb = []

for n in [1500, 1000, 500, 200]:
    for d in [20, 10,5]:
        for lr in [0.05, 0.1, 0.2]:
            pipe_xgb.set_params(
                clf__n_estimators=n,
                clf__max_depth=d,
                clf__learning_rate=lr
            )
            pipe_xgb.fit(X_tune, y_tune)

            print(f"\nTrying: n_estimators={n}, max_depth={d},
learning_rate={lr}")
```

```
            evaluate(pipe_xgb, X_val, y_val_orig)

            score = f1_val(pipe_xgb)
            results_xgb.append({
                'n_estimators': n,
                'max_depth': d,
                'learning_rate': lr,
                'f1_val': score
            })

            if score > best_score:
                best_score = score
                best_params_xgb = {'n_estimators': n, 'max_depth': d,
'learning_rate': lr}

print("\n=== XGB Grid Search Completed ===")
print("Best params based on F1 on validation set:", best_params_xgb)
```

```
Trying: n_estimators=1500, max_depth=20, learning_rate=0.05

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7744   55]
 [  95  106]]
Accuracy:       0.98125
Precision(0):   0.6583850931677019
Recall(0):      0.527363184079602
Specificity:    0.9929478138222849
F1(0):          0.585635359116022

Trying: n_estimators=1500, max_depth=20, learning_rate=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7746   53]
 [  96  105]]
Accuracy:       0.981375
Precision(0):   0.6645569620253164
Recall(0):      0.5223880597014925
Specificity:    0.99320425695602
F1(0):          0.584958217270195

Trying: n_estimators=1500, max_depth=20, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7742   57]
 [  98  103]]
Accuracy:       0.980625
```

```
Precision(0):   0.64375
Recall(0):      0.5124378109452736
Specificity:    0.9926913706885498
F1(0):          0.5706371191135734

Trying: n_estimators=1500, max_depth=10, learning_rate=0.05

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7745   54]
 [ 100  101]]
Accuracy:       0.98075
Precision(0):   0.6516129032258065
Recall(0):      0.5024875621890548
Specificity:    0.9930760353891525
F1(0):          0.5674157303370787

Trying: n_estimators=1500, max_depth=10, learning_rate=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7744   55]
 [  98  103]]
Accuracy:       0.980875
Precision(0):   0.6518987341772152
Recall(0):      0.5124378109452736
Specificity:    0.9929478138222849
F1(0):          0.5738161559888579

Trying: n_estimators=1500, max_depth=10, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7750   49]
 [  98  103]]
Accuracy:       0.981625
Precision(0):   0.6776315789473685
Recall(0):      0.5124378109452736
Specificity:    0.9937171432234901
F1(0):          0.5835694050991501

Trying: n_estimators=1500, max_depth=5, learning_rate=0.05

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7667  132]
 [  80  121]]
Accuracy:       0.9735
Precision(0):   0.4782608695652174
Recall(0):      0.6019900497512438
```

```
Specificity:    0.9830747531734838
F1(0):          0.5330396475770925

Trying: n_estimators=1500, max_depth=5, learning_rate=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7710   89]
 [  95  106]]
Accuracy:       0.977
Precision(0):   0.5435897435897435
Recall(0):      0.527363184079602
Specificity:    0.9885882805487883
F1(0):          0.5353535353535354

Trying: n_estimators=1500, max_depth=5, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7724   75]
 [  91  110]]
Accuracy:       0.97925
Precision(0):   0.5945945945945946
Recall(0):      0.5472636815920398
Specificity:    0.990383382484934
F1(0):          0.5699481865284974

Trying: n_estimators=1000, max_depth=20, learning_rate=0.05

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7745   54]
 [  95  106]]
Accuracy:       0.981375
Precision(0):   0.6625
Recall(0):      0.527363184079602
Specificity:    0.9930760353891525
F1(0):          0.5872576177285319

Trying: n_estimators=1000, max_depth=20, learning_rate=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7745   54]
 [  96  105]]
Accuracy:       0.98125
Precision(0):   0.660377358490566
Recall(0):      0.5223880597014925
Specificity:    0.9930760353891525
F1(0):          0.5833333333333334
```

```
Trying: n_estimators=1000, max_depth=20, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7744   55]
 [  97  104]]
Accuracy:       0.981
Precision(0):   0.6540880503144654
Recall(0):      0.5174129353233831
Specificity:    0.9929478138222849
F1(0):          0.5777777777777777

Trying: n_estimators=1000, max_depth=10, learning_rate=0.05

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7740   59]
 [  99  102]]
Accuracy:       0.98025
Precision(0):   0.6335403726708074
Recall(0):      0.5074626865671642
Specificity:    0.9924349275548148
F1(0):          0.56353591160221

Trying: n_estimators=1000, max_depth=10, learning_rate=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7739   60]
 [  98  103]]
Accuracy:       0.98025
Precision(0):   0.6319018404907976
Recall(0):      0.5124378109452736
Specificity:    0.9923067059879471
F1(0):          0.5659340659340659

Trying: n_estimators=1000, max_depth=10, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7750   49]
 [ 100  101]]
Accuracy:       0.981375
Precision(0):   0.6733333333333333
Recall(0):      0.5024875621890548
Specificity:    0.9937171432234901
F1(0):          0.5754985754985755

Trying: n_estimators=1000, max_depth=5, learning_rate=0.05
```

```
=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7601  198]
 [  68  133]]
Accuracy:       0.96675
Precision(0):   0.40181268882175225
Recall(0):      0.6616915422885572
Specificity:    0.9746121297602257
F1(0):          0.5

Trying: n_estimators=1000, max_depth=5, learning_rate=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7689  110]
 [  90  111]]
Accuracy:       0.975
Precision(0):   0.502262443438914
Recall(0):      0.5522388059701493
Specificity:    0.9858956276445698
F1(0):          0.5260663507109005

Trying: n_estimators=1000, max_depth=5, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7722   77]
 [  90  111]]
Accuracy:       0.979125
Precision(0):   0.5904255319148937
Recall(0):      0.5522388059701493
Specificity:    0.9901269393511989
F1(0):          0.570694087403599

Trying: n_estimators=500, max_depth=20, learning_rate=0.05

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7741   58]
 [  94  107]]
Accuracy:       0.981
Precision(0):   0.6484848484848484
Recall(0):      0.5323383084577115
Specificity:    0.9925631491216823
F1(0):          0.5846994535519126

Trying: n_estimators=500, max_depth=20, learning_rate=0.1

=== clf ===
```

```
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7745   54]
 [  95  106]]
Accuracy:       0.981375
Precision(0):   0.6625
Recall(0):      0.527363184079602
Specificity:    0.9930760353891525
F1(0):          0.5872576177285319

Trying: n_estimators=500, max_depth=20, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7743   56]
 [  97  104]]
Accuracy:       0.980875
Precision(0):   0.65
Recall(0):      0.5174129353233831
Specificity:    0.9928195922554174
F1(0):          0.5761772853185596

Trying: n_estimators=500, max_depth=10, learning_rate=0.05

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7735   64]
 [  97  104]]
Accuracy:       0.979875
Precision(0):   0.6190476190476191
Recall(0):      0.5174129353233831
Specificity:    0.991793819720477
F1(0):          0.5636856368563685

Trying: n_estimators=500, max_depth=10, learning_rate=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7738   61]
 [  96  105]]
Accuracy:       0.980375
Precision(0):   0.6325301204819277
Recall(0):      0.5223880597014925
Specificity:    0.9921784844210796
F1(0):          0.5722070844686649

Trying: n_estimators=500, max_depth=10, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7748   51]
```

```
 [  99  102]]
Accuracy:       0.98125
Precision(0):   0.6666666666666666
Recall(0):      0.5074626865671642
Specificity:    0.9934607000897551
F1(0):          0.576271186440678

Trying: n_estimators=500, max_depth=5, learning_rate=0.05

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7430  369]
 [  52  149]]
Accuracy:       0.947375
Precision(0):   0.2876447876447876
Recall(0):      0.7412935323383084
Specificity:    0.9526862418258751
F1(0):          0.4144645340751043

Trying: n_estimators=500, max_depth=5, learning_rate=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7602  197]
 [  69  132]]
Accuracy:       0.96675
Precision(0):   0.4012158054711246
Recall(0):      0.6567164179104478
Specificity:    0.9747403513270932
F1(0):          0.4981132075471698

Trying: n_estimators=500, max_depth=5, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7702   97]
 [  86  115]]
Accuracy:       0.977125
Precision(0):   0.5424528301886793
Recall(0):      0.572139303482587
Specificity:    0.9875625080138479
F1(0):          0.5569007263922519

Trying: n_estimators=200, max_depth=20, learning_rate=0.05

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7730   69]
 [  93  108]]
Accuracy:       0.97975
```

```
Precision(0):   0.6101694915254238
Recall(0):      0.5373134328358209
Specificity:    0.9911527118861393
F1(0):          0.5714285714285714

Trying: n_estimators=200, max_depth=20, learning_rate=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7741   58]
 [  94  107]]
Accuracy:       0.981
Precision(0):   0.6484848484848484
Recall(0):      0.5323383084577115
Specificity:    0.9925631491216823
F1(0):          0.5846994535519126

Trying: n_estimators=200, max_depth=20, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7746   53]
 [  96  105]]
Accuracy:       0.981375
Precision(0):   0.6645569620253164
Recall(0):      0.5223880597014925
Specificity:    0.99320425695602
F1(0):          0.584958217270195

Trying: n_estimators=200, max_depth=10, learning_rate=0.05

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7640  159]
 [  82  119]]
Accuracy:       0.969875
Precision(0):   0.42805755395683454
Recall(0):      0.5920398009950248
Specificity:    0.97961277086806
F1(0):          0.4968684759916493

Trying: n_estimators=200, max_depth=10, learning_rate=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7718   81]
 [  92  109]]
Accuracy:       0.978375
Precision(0):   0.5736842105263158
Recall(0):      0.5422885572139303
```

```
Specificity:    0.9896140530837286
F1(0):          0.5575447570332481

Trying: n_estimators=200, max_depth=10, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7744   55]
 [  98  103]]
Accuracy:       0.980875
Precision(0):   0.6518987341772152
Recall(0):      0.5124378109452736
Specificity:    0.9929478138222849
F1(0):          0.5738161559888579

Trying: n_estimators=200, max_depth=5, learning_rate=0.05

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7235  564]
 [  38  163]]
Accuracy:       0.92475
Precision(0):   0.22420907840440166
Recall(0):      0.8109452736318408
Specificity:    0.9276830362867035
F1(0):          0.35129310344827586

Trying: n_estimators=200, max_depth=5, learning_rate=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7396  403]
 [  49  152]]
Accuracy:       0.9435
Precision(0):   0.27387387387387385
Recall(0):      0.7562189054726368
Specificity:    0.9483267085523786
F1(0):          0.4021164021164021

Trying: n_estimators=200, max_depth=5, learning_rate=0.2

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7583  216]
 [  68  133]]
Accuracy:       0.9645
Precision(0):   0.38108882521489973
Recall(0):      0.6616915422885572
Specificity:    0.9723041415566098
F1(0):          0.48363636363636364
```

```
=== XGB Grid Search Completed ===
Best params based on F1 on validation set: {'n_estimators': 1000,
'max_depth': 20, 'learning_rate': 0.05}
```

```
df_results = pd.DataFrame(results_xgb)
print(df_results.sort_values('f1_val', ascending=False))
```

```
    n_estimators  max_depth  learning_rate    f1_val
9           1000         20           0.05  0.587258
19           500         20           0.10  0.587258
0           1500         20           0.05  0.585635
29           200         20           0.20  0.584958
1           1500         20           0.10  0.584958
28           200         20           0.10  0.584699
18           500         20           0.05  0.584699
5           1500         10           0.20  0.583569
10          1000         20           0.10  0.583333
11          1000         20           0.20  0.577778
23           500         10           0.20  0.576271
20           500         20           0.20  0.576177
14          1000         10           0.20  0.575499
32           200         10           0.20  0.573816
4           1500         10           0.10  0.573816
22           500         10           0.10  0.572207
27           200         20           0.05  0.571429
17          1000          5           0.20  0.570694
2           1500         20           0.20  0.570637
8           1500          5           0.20  0.569948
3           1500         10           0.05  0.567416
13          1000         10           0.10  0.565934
21           500         10           0.05  0.563686
12          1000         10           0.05  0.563536
31           200         10           0.10  0.557545
26           500          5           0.20  0.556901
7           1500          5           0.10  0.535354
6           1500          5           0.05  0.533040
16          1000          5           0.10  0.526066
15          1000          5           0.05  0.500000
25           500          5           0.10  0.498113
30           200         10           0.05  0.496868
35           200          5           0.20  0.483636
24           500          5           0.05  0.414465
34           200          5           0.10  0.402116
33           200          5           0.05  0.351293
```

```
best_params_xgb = {'n_estimators': 500, 'max_depth': 10,
'learning_rate': 0.2}
print("\n=== XGB Grid Search Completed ===")
```

```
print("Best params based on confusion matrix on validation set:",
best_params_xgb)
```

```
=== XGB Grid Search Completed ===
Best params based on confusion matrix on validation set:
{'n_estimators': 500, 'max_depth': 10, 'learning_rate': 0.2}
```

```
# 1. Set up the MLP pipeline
pipe_mlp = Pipeline([
    ('pre', preprocessor),
    ('clf', MLPClassifier(
        random_state=42,
        max_iter=240,
        tol=1e-4,
        verbose=False
    ))
])

best_score = -1
best_params_mlp = {}
results_mlp = []

for hidden_layers in [(10,), (20,), (20, 10), (30, 20, 10)]:
    for alpha in [1e-4, 1e-3]:
        for lr in [5e-3, 1e-2, 1e-1]:
            pipe_mlp.set_params(
                clf__hidden_layer_sizes=hidden_layers,
                clf__alpha=alpha,
                clf__learning_rate_init=lr,
                clf__learning_rate='constant'
            )


            pipe_mlp.fit(X_tune, y_tune)

            print(f"\nTrying: layers={hidden_layers}, alpha={alpha},
lr_init={lr}")
            evaluate(pipe_mlp, X_val, y_val_orig)


            score = f1_val(pipe_mlp)
            results_mlp.append({
                'hidden_layers': hidden_layers,
                'alpha': alpha,
                'learning_rate_init': lr,
                'f1_val': score
            })
```

```
            if score > best_score:
                best_score = score
                best_params_mlp = {
                    'hidden_layer_sizes': hidden_layers,
                    'alpha': alpha,
                    'learning_rate_init': lr
                }

print("\n=== MLP Grid Search Completed ===")
print("Best params based on F1 on validation set:", best_params_mlp)
```

```
Trying: layers=(10,), alpha=0.0001, lr_init=0.005

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7773   25]
 [  98  104]]
Accuracy:       0.984625
Precision(0):   0.8062015503875969
Recall(0):      0.5148514851485149
Specificity:    0.9967940497563478
F1(0):          0.6283987915407855

Trying: layers=(10,), alpha=0.0001, lr_init=0.01

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7776   22]
 [  96  106]]
Accuracy:       0.98525
Precision(0):   0.828125
Recall(0):      0.5247524752475248
Specificity:    0.997178763785586
F1(0):          0.6424242424242425

Trying: layers=(10,), alpha=0.0001, lr_init=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7760   38]
 [  92  110]]
Accuracy:       0.98375
Precision(0):   0.7432432432432432
Recall(0):      0.5445544554455446
Specificity:    0.9951269556296486
F1(0):          0.6285714285714286

Trying: layers=(10,), alpha=0.001, lr_init=0.005
```

```
=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7773   25]
 [  98  104]]
Accuracy:       0.984625
Precision(0):   0.8062015503875969
Recall(0):      0.5148514851485149
Specificity:    0.9967940497563478
F1(0):          0.6283987915407855

Trying: layers=(10,), alpha=0.001, lr_init=0.01

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7781   17]
 [  97  105]]
Accuracy:       0.98575
Precision(0):   0.860655737704918
Recall(0):      0.5198019801980198
Specificity:    0.9978199538343165
F1(0):          0.6481481481481481

Trying: layers=(10,), alpha=0.001, lr_init=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7777   21]
 [  99  103]]
Accuracy:       0.985
Precision(0):   0.8306451612903226
Recall(0):      0.5099009900990099
Specificity:    0.9973070017953322
F1(0):          0.6319018404907976

Trying: layers=(20,), alpha=0.0001, lr_init=0.005

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7778   20]
 [  95  107]]
Accuracy:       0.985625
Precision(0):   0.84251968503937
Recall(0):      0.5297029702970297
Specificity:    0.9974352398050782
F1(0):          0.6504559270516718

Trying: layers=(20,), alpha=0.0001, lr_init=0.01

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
```

```
 [[7776   22]
 [  95  107]]
Accuracy:       0.985375
Precision(0):   0.8294573643410853
Recall(0):      0.5297029702970297
Specificity:    0.997178763785586
F1(0):          0.6465256797583081

Trying: layers=(20,), alpha=0.0001, lr_init=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7777   21]
 [  97  105]]
Accuracy:       0.98525
Precision(0):   0.8333333333333334
Recall(0):      0.5198019801980198
Specificity:    0.9973070017953322
F1(0):          0.6402439024390244

Trying: layers=(20,), alpha=0.001, lr_init=0.005

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7784   14]
 [ 102  100]]
Accuracy:       0.9855
Precision(0):   0.8771929824561403
Recall(0):      0.49504950495049505
Specificity:    0.9982046678635548
F1(0):          0.6329113924050633

Trying: layers=(20,), alpha=0.001, lr_init=0.01

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7775   23]
 [  93  109]]
Accuracy:       0.9855
Precision(0):   0.8257575757575758
Recall(0):      0.5396039603960396
Specificity:    0.99705052577584
F1(0):          0.6526946107784432

Trying: layers=(20,), alpha=0.001, lr_init=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7780   18]
 [ 114   88]]
```

```
Accuracy:       0.9835
Precision(0):   0.8301886792452831
Recall(0):      0.43564356435643564
Specificity:    0.9976917158245704
F1(0):          0.5714285714285714

Trying: layers=(20, 10), alpha=0.0001, lr_init=0.005

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7781   17]
 [  97  105]]
Accuracy:       0.98575
Precision(0):   0.860655737704918
Recall(0):      0.5198019801980198
Specificity:    0.9978199538343165
F1(0):          0.6481481481481481

Trying: layers=(20, 10), alpha=0.0001, lr_init=0.01

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7772   26]
 [  96  106]]
Accuracy:       0.98475
Precision(0):   0.803030303030303
Recall(0):      0.5247524752475248
Specificity:    0.9966658117466017
F1(0):          0.6347305389221557

Trying: layers=(20, 10), alpha=0.0001, lr_init=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7782   16]
 [ 109   93]]
Accuracy:       0.984375
Precision(0):   0.8532110091743119
Recall(0):      0.4603960396039604
Specificity:    0.9979481918440626
F1(0):          0.5980707395498392

Trying: layers=(20, 10), alpha=0.001, lr_init=0.005

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7782   16]
 [ 120   82]]
Accuracy:       0.983
Precision(0):   0.8367346938775511
```

```
Recall(0):      0.40594059405940597
Specificity:    0.9979481918440626
F1(0):          0.5466666666666666

Trying: layers=(20, 10), alpha=0.001, lr_init=0.01

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7790    8]
 [ 132   70]]
Accuracy:       0.9825
Precision(0):   0.8974358974358975
Recall(0):      0.3465346534653465
Specificity:    0.9989740959220312
F1(0):          0.5

Trying: layers=(20, 10), alpha=0.001, lr_init=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7792    6]
 [ 123   79]]
Accuracy:       0.983875
Precision(0):   0.9294117647058824
Recall(0):      0.3910891089108911
Specificity:    0.9992305719415234
F1(0):          0.5505226480836237

Trying: layers=(30, 20, 10), alpha=0.0001, lr_init=0.005

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7760   38]
 [  95  107]]
Accuracy:       0.983375
Precision(0):   0.7379310344827587
Recall(0):      0.5297029702970297
Specificity:    0.9951269556296486
F1(0):          0.6167146974063401

Trying: layers=(30, 20, 10), alpha=0.0001, lr_init=0.01

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7765   33]
 [  93  109]]
Accuracy:       0.98425
Precision(0):   0.7676056338028169
Recall(0):      0.5396039603960396
Specificity:    0.9957681456783791
```

```
F1(0):          0.6337209302325582

Trying: layers=(30, 20, 10), alpha=0.0001, lr_init=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7764   34]
 [ 103   99]]
Accuracy:       0.982875
Precision(0):   0.7443609022556391
Recall(0):      0.4900990099009901
Specificity:    0.995639907668633
F1(0):          0.591044776119403

Trying: layers=(30, 20, 10), alpha=0.001, lr_init=0.005

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7767   31]
 [  93  109]]
Accuracy:       0.9845
Precision(0):   0.7785714285714286
Recall(0):      0.5396039603960396
Specificity:    0.9960246216978712
F1(0):          0.6374269005847953

Trying: layers=(30, 20, 10), alpha=0.001, lr_init=0.01

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7778   20]
 [  95  107]]
Accuracy:       0.985625
Precision(0):   0.84251968503937
Recall(0):      0.5297029702970297
Specificity:    0.9974352398050782
F1(0):          0.6504559270516718

Trying: layers=(30, 20, 10), alpha=0.001, lr_init=0.1

=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[7757   41]
 [  92  110]]
Accuracy:       0.983375
Precision(0):   0.7284768211920529
Recall(0):      0.5445544554455446
Specificity:    0.9947422416004104
F1(0):          0.623229461756374
```

```
=== MLP Grid Search Completed ===
Best params based on F1 on validation set: {'hidden_layer_sizes':
(20,), 'alpha': 0.001, 'learning_rate_init': 0.01}
```

```
df_results_mlp = pd.DataFrame(results_mlp)
print(df_results_mlp.sort_values('f1_val', ascending=False))
```

```
   hidden_layers   alpha  learning_rate_init    f1_val
10         (20,)  0.0010               0.010  0.652695
22  (30, 20, 10)  0.0010               0.010  0.650456
6          (20,)  0.0001               0.005  0.650456
12      (20, 10)  0.0001               0.005  0.648148
4          (10,)  0.0010               0.010  0.648148
7          (20,)  0.0001               0.010  0.646526
1          (10,)  0.0001               0.010  0.642424
8          (20,)  0.0001               0.100  0.640244
21  (30, 20, 10)  0.0010               0.005  0.637427
13      (20, 10)  0.0001               0.010  0.634731
19  (30, 20, 10)  0.0001               0.010  0.633721
9          (20,)  0.0010               0.005  0.632911
5          (10,)  0.0010               0.100  0.631902
2          (10,)  0.0001               0.100  0.628571
0          (10,)  0.0001               0.005  0.628399
3          (10,)  0.0010               0.005  0.628399
23  (30, 20, 10)  0.0010               0.100  0.623229
18  (30, 20, 10)  0.0001               0.005  0.616715
14      (20, 10)  0.0001               0.100  0.598071
20  (30, 20, 10)  0.0001               0.100  0.591045
11         (20,)  0.0010               0.100  0.571429
17      (20, 10)  0.0010               0.100  0.550523
15      (20, 10)  0.0010               0.005  0.546667
16      (20, 10)  0.0010               0.010  0.500000
```

```
print("\n=== MLP Grid Search Completed ===")
print("Best params based on F1 on validation set:", best_params_mlp)
```

```
=== MLP Grid Search Completed ===
Best params based on F1 on validation set: {'hidden_layer_sizes':
(20,), 'alpha': 0.001, 'learning_rate_init': 0.01}
```

```
import random
from sklearn.preprocessing import FunctionTransformer
from scikeras.wrappers       import KerasClassifier
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, SimpleRNN, Dropout, Dense
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import confusion_matrix, f1_score
```

```
X_tune_proc = preprocessor.fit_transform(X_tune)
X_val_proc  = preprocessor.transform(X_val)
n_features  = X_tune_proc.shape[1]

reshape = lambda X: X.reshape(-1, 1, X.shape[1])

def build_rnn_model(units=32, dropout_rate=0.2, learning_rate=1e-3):
    m = Sequential([
        Input(shape=(1, n_features)),
        SimpleRNN(units, dropout=dropout_rate),
        Dense(1, activation='sigmoid')
    ])
    m.compile(optimizer=Adam(learning_rate=learning_rate),
              loss='binary_crossentropy')
    return m

param_dist = {
    'units':         [16, 32, 64, 128],
    'dropout_rate':  [0.0, 0.2, 0.1],
    'learning_rate': [1e-2, 1e-3],
    'epochs':        [20, 50, 100],
    'batch_size':    [32, 64, 128],
}

best_score, best_params = -1, {}
results = []
n_iter = 20

for i in range(n_iter):
    sample = {k: random.choice(v) for k, v in param_dist.items()}

    clf = KerasClassifier(
        model=build_rnn_model,
        units=sample['units'],
        dropout_rate=sample['dropout_rate'],
        learning_rate=sample['learning_rate'],
        epochs=sample['epochs'],
        batch_size=sample['batch_size'],
        verbose=0,
        random_state=42
    )

    X_train_3d = reshape(X_tune_proc)
    clf.fit(X_train_3d, y_tune)

    X_val_3d    = reshape(X_val_proc)
    y_val_pred  = clf.predict(X_val_3d)

    score = f1_score(y_val, y_val_pred, pos_label=1)
    cm    = confusion_matrix(y_val, y_val_pred)
```

```
    print(f"\nTrial {i+1}/{n_iter}: {sample}")
    print("Confusion matrix:\n", cm)
    print(f"F1 = {score:.4f}")

    results.append({**sample, 'f1_val': score})
    if score > best_score:
        best_score, best_params = score, sample

print("\n=== Randomized RNN Search Complete ===")
print("Best F1:", best_score)
print("Best params:", best_params)
```

```
Trial 1/20: {'units': 16, 'dropout_rate': 0.0, 'learning_rate': 0.01,
'epochs': 100, 'batch_size': 32}
Confusion matrix:
 [[7776   22]
 [ 100  102]]
F1 = 0.6258

Trial 2/20: {'units': 16, 'dropout_rate': 0.1, 'learning_rate': 0.01,
'epochs': 20, 'batch_size': 64}
Confusion matrix:
 [[7792    6]
 [ 122   80]]
F1 = 0.5556

Trial 3/20: {'units': 32, 'dropout_rate': 0.1, 'learning_rate': 0.001,
'epochs': 100, 'batch_size': 128}
Confusion matrix:
 [[7787   11]
 [ 111   91]]
F1 = 0.5987

Trial 4/20: {'units': 16, 'dropout_rate': 0.0, 'learning_rate': 0.001,
'epochs': 50, 'batch_size': 128}
Confusion matrix:
 [[7773   25]
 [ 100  102]]
F1 = 0.6201

Trial 5/20: {'units': 32, 'dropout_rate': 0.0, 'learning_rate': 0.01,
'epochs': 20, 'batch_size': 64}
Confusion matrix:
 [[7778   20]
 [ 100  102]]
F1 = 0.6296

Trial 6/20: {'units': 128, 'dropout_rate': 0.1, 'learning_rate':
```

```
0.001, 'epochs': 50, 'batch_size': 64}
Confusion matrix:
 [[7789    9]
 [ 119   83]]
F1 = 0.5646

Trial 7/20: {'units': 128, 'dropout_rate': 0.1, 'learning_rate':
0.001, 'epochs': 100, 'batch_size': 32}
Confusion matrix:
 [[7789    9]
 [ 115   87]]
F1 = 0.5839

Trial 8/20: {'units': 32, 'dropout_rate': 0.2, 'learning_rate': 0.01,
'epochs': 20, 'batch_size': 128}
Confusion matrix:
 [[7798    0]
 [ 182   20]]
F1 = 0.1802

Trial 9/20: {'units': 16, 'dropout_rate': 0.2, 'learning_rate': 0.001,
'epochs': 50, 'batch_size': 128}
Confusion matrix:
 [[7793    5]
 [ 166   36]]
F1 = 0.2963

Trial 10/20: {'units': 64, 'dropout_rate': 0.2, 'learning_rate':
0.001, 'epochs': 20, 'batch_size': 64}
Confusion matrix:
 [[7796    2]
 [ 162   40]]
F1 = 0.3279

Trial 11/20: {'units': 32, 'dropout_rate': 0.2, 'learning_rate':
0.001, 'epochs': 20, 'batch_size': 128}
Confusion matrix:
 [[7794    4]
 [ 156   46]]
F1 = 0.3651

Trial 12/20: {'units': 128, 'dropout_rate': 0.0, 'learning_rate':
0.001, 'epochs': 100, 'batch_size': 64}
Confusion matrix:
 [[7776   22]
 [  98  104]]
F1 = 0.6341

Trial 13/20: {'units': 64, 'dropout_rate': 0.0, 'learning_rate':
0.001, 'epochs': 100, 'batch_size': 64}
```

```
Confusion matrix:
 [[7774   24]
 [  97  105]]
F1 = 0.6344

Trial 14/20: {'units': 128, 'dropout_rate': 0.0, 'learning_rate':
0.01, 'epochs': 100, 'batch_size': 32}
Confusion matrix:
 [[7774   24]
 [ 101  101]]
F1 = 0.6177

Trial 15/20: {'units': 128, 'dropout_rate': 0.1, 'learning_rate':
0.001, 'epochs': 50, 'batch_size': 64}
Confusion matrix:
 [[7789    9]
 [ 119   83]]
F1 = 0.5646

Trial 16/20: {'units': 16, 'dropout_rate': 0.1, 'learning_rate': 0.01,
'epochs': 100, 'batch_size': 32}
Confusion matrix:
 [[7797    1]
 [ 162   40]]
F1 = 0.3292

Trial 17/20: {'units': 16, 'dropout_rate': 0.2, 'learning_rate': 0.01,
'epochs': 100, 'batch_size': 32}
Confusion matrix:
 [[7798    0]
 [ 197    5]]
F1 = 0.0483

Trial 18/20: {'units': 32, 'dropout_rate': 0.1, 'learning_rate': 0.01,
'epochs': 20, 'batch_size': 64}
Confusion matrix:
 [[7791    7]
 [ 122   80]]
F1 = 0.5536

Trial 19/20: {'units': 64, 'dropout_rate': 0.2, 'learning_rate':
0.001, 'epochs': 20, 'batch_size': 64}
Confusion matrix:
 [[7796    2]
 [ 162   40]]
F1 = 0.3279

Trial 20/20: {'units': 32, 'dropout_rate': 0.2, 'learning_rate':
0.001, 'epochs': 50, 'batch_size': 32}
Confusion matrix:
```

```
 [[7796    2]
 [ 163   39]]
F1 = 0.3210

=== Randomized RNN Search Complete ===
Best F1: 0.6344410876132931
Best params: {'units': 64, 'dropout_rate': 0.0, 'learning_rate':
0.001, 'epochs': 100, 'batch_size': 64}
```

```
df_results_rnn = pd.DataFrame(results)
print(df_results_rnn.sort_values('f1_val', ascending=False))
```

```
    units  dropout_rate  learning_rate  epochs  batch_size    f1_val
12     64           0.0          0.001     100          64  0.634441
11    128           0.0          0.001     100          64  0.634146
4      32           0.0          0.010      20          64  0.629630
0      16           0.0          0.010     100          32  0.625767
3      16           0.0          0.001      50         128  0.620061
13    128           0.0          0.010     100          32  0.617737
2      32           0.1          0.001     100         128  0.598684
6     128           0.1          0.001     100          32  0.583893
5     128           0.1          0.001      50          64  0.564626
14    128           0.1          0.001      50          64  0.564626
1      16           0.1          0.010      20          64  0.555556
17     32           0.1          0.010      20          64  0.553633
10     32           0.2          0.001      20         128  0.365079
15     16           0.1          0.010     100          32  0.329218
9      64           0.2          0.001      20          64  0.327869
18     64           0.2          0.001      20          64  0.327869
19     32           0.2          0.001      50          32  0.320988
8      16           0.2          0.001      50         128  0.296296
7      32           0.2          0.010      20         128  0.180180
16     16           0.2          0.010     100          32  0.048309
```

```
best_params = {
    'units': 32,
    'dropout_rate': 0.1,
    'learning_rate': 0.001,
    'epochs': 100,
    'batch_size': 128
}
print("\n=== Randomized RNN Search Complete ===")
print("Best params:", best_params)
```

```
=== Randomized RNN Search Complete ===
Best params: {'units': 32, 'dropout_rate': 0.1, 'learning_rate':
0.001, 'epochs': 100, 'batch_size': 128}
```

```
pd.concat([X_sub, y_sub.rename('Default_bin')], axis=1)\
  .to_csv("subset_train_40000forXGBMLPRNN.csv", index=False)
```

```
best_params_xgb
```

```
{'n_estimators': 500, 'max_depth': 10, 'learning_rate': 0.2}
```

```
n_neg_full = (y_train_full == 0).sum()
n_pos_full = (y_train_full == 1).sum()
scale_pos_weight_full = n_neg_full / n_pos_full

# 10. Train final models on full 2000 data
# -- XGB
final_xgb = Pipeline([
    ('pre', preprocessor),
    ('clf', XGBClassifier(
        use_label_encoder=False, eval_metric='logloss',
random_state=42,
        scale_pos_weight=scale_pos_weight_full,
        **best_params_xgb
    ))
])
final_xgb.fit(X_train_full, y_train_full)
```

```
c:\Users\z8360\anaconda3\envs\tf_env\lib\site-packages\xgboost\
core.py:158: UserWarning: [06:07:08] WARNING: C:\buildkite-agent\
builds\buildkite-windows-cpu-autoscaling-group-i-08cbc0333d8d4aae1-1\
xgboost\xgboost-ci-windows\src\learner.cc:740:
Parameters: { "use_label_encoder" } are not used.

  warnings.warn(smsg, UserWarning)
```

```
Pipeline(steps=[('pre',
                 ColumnTransformer(transformers=[('num',
StandardScaler(),
                                                  ['ORIG_AMT',
'ORIG_TRM',
                                                   'NUM_BO', 'DTI',

'reperformance_flag',
                                                   'Loan.Age',
'FicoBkt',
                                                   'LAST_VAL',
'MLTV']),
                                                 ('cat',

OneHotEncoder(handle_unknown='ignore',

sparse_output=False),
                                                  ['PROP_TYP'])])),
                ('clf',
                 XGBClassifier(base_score=None, booster=None,
callbacks=None,
                               colsample_bylevel=None,...
```

```
                                feature_types=None, gamma=None,
grow_policy=None,
                                importance_type=None,
                                interaction_constraints=None,
learning_rate=0.2,
                                max_bin=None, max_cat_threshold=None,
                                max_cat_to_onehot=None,
max_delta_step=None,
                                max_depth=10, max_leaves=None,
                                min_child_weight=None, missing=nan,
                                monotone_constraints=None,
multi_strategy=None,
                                n_estimators=500, n_jobs=None,
                                num_parallel_tree=None,
random_state=42, ...))])
```

```
evaluate(final_xgb, X_test, y_test_orig)
```

```
=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[702964   7307]
 [ 17017   5754]]
Accuracy:       0.9668177266786896
Precision(0):   0.4405481969221346
Recall(0):      0.2526898247771288
Specificity:    0.9897123773883489
F1(0):          0.32116543871399866
```

```
best_params_mlp
```

```
{'hidden_layer_sizes': (20,), 'alpha': 0.001, 'learning_rate_init':
0.01}
```

```
# -- MLP
final_mlp = Pipeline([
    ('pre', preprocessor),
    ('clf', MLPClassifier(
        max_iter=200, random_state=42, **best_params_mlp
    ))
])
final_mlp.fit(X_train_full, y_train_full)
```

```
Pipeline(steps=[('pre',
                 ColumnTransformer(transformers=[('num',
StandardScaler(),
                                                  ['ORIG_AMT',
'ORIG_TRM',
                                                   'NUM_BO', 'DTI',

'reperformance_flag',
```

```
                                                            'Loan.Age',
'FicoBkt',
                                                            'LAST_VAL',
'MLTV']),
                                                        ('cat',

OneHotEncoder(handle_unknown='ignore',

sparse_output=False),
                                                         ['PROP_TYP'])])),
                ('clf',
                 MLPClassifier(alpha=0.001, hidden_layer_sizes=(20,),
                               learning_rate_init=0.01,
random_state=42))])
```

```
best_params
```

```
{'units': 32,
 'dropout_rate': 0.1,
 'learning_rate': 0.001,
 'epochs': 100,
 'batch_size': 128}
```

```
best_params_rnn = best_params.copy()
```

```
# -- RNN
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow import keras
try:
    from scikeras.wrappers import KerasClassifier
    print("Using KerasClassifier from scikeras.wrappers")
except ImportError:
    try:
        from tensorflow.keras.wrappers.scikit_learn import
KerasClassifier
        print("Using KerasClassifier from
tensorflow.keras.wrappers.scikit_learn")
    except ImportError:
        print("ERROR: KerasClassifier wrapper not found. Please
install scikeras (`pip install scikeras`).")
        KerasClassifier = None
from sklearn.utils import class_weight
from sklearn.metrics import confusion_matrix
import time
import os
import random
import sys
```

```
SEED = 42
os.environ['PYTHONHASHSEED']=str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("--- Running Final RNN Training/Evaluation (User's Structure -
Fixed Again) ---")

if KerasClassifier is None: raise ImportError("KerasClassifier not
found.")

# 1. Instantiate KerasClassifier Wrapper
print("Instantiating KerasClassifier...")
final_keras = KerasClassifier(
    model=build_rnn_model,
    units=best_params_rnn['units'],
    dropout_rate=best_params_rnn['dropout_rate'],
    learning_rate=best_params_rnn['learning_rate'],
    epochs=best_params_rnn['epochs'],
    batch_size=best_params_rnn['batch_size'],
    verbose=0,

)

print("Preprocessing and reshaping training data...")
X_train_3d = preprocessor.transform(X_train_full).reshape(-1, 1,
n_features)

# 3. Calculate class_weights_dict
print("Calculating class weights for final training...")
try:
    class_weights_list_final = class_weight.compute_class_weight(
        'balanced', classes=np.unique(y_train_full), y=y_train_full
    )
    class_weights_dict = dict(enumerate(class_weights_list_final))
    print(f"Calculated Class Weights (from y_train_full):
{class_weights_dict}")
except Exception as e:
     print(f"Warning: Could not calculate class weights: {e}.
Proceeding without them.")
     class_weights_dict = None


print(f"Fitting final_keras model for {best_params_rnn['epochs']}
epochs...")
try:
    final_keras.fit(
        X_train_3d,
        y_train_full,
```

```
        class_weight=class_weights_dict
    )
    print("Model fitting complete.")
except Exception as e:
    print(f"An error occurred during final_keras.fit: {e}")
    print("Stopping script because fitting failed.")
    raise


#  Preprocess and Reshape Test Data
print("Preprocessing and reshaping test data...")
X_test_3d = preprocessor.transform(X_test).reshape(-1, 1, n_features)

#  Define the Evaluation Function
def evaluate_rnn(model, X, y_orig, n_features):
    """
    Evaluates RNN using the user's specific structure,
    corrected to use predict_proba for consistent evaluation.
    """
    print("\n=== RNN Evaluation (User's Function - Revised) ===")
    try:
        print("  Preprocessing/reshaping data inside evaluate_rnn...")
        X_proc = preprocessor.transform(X).reshape(-1, 1, n_features)
        print(f"  Processed data shape: {X_proc.shape}")

        print("  Generating probabilities using
model.predict_proba...")
        if not hasattr(model, 'predict_proba'):
             print("  ERROR: Model object does not have
'predict_proba' method!")
             y_prob_pos = model.predict(X_proc).ravel()
             print("  Warning: predict_proba not found, falling back
to predict.")
        else:
            y_prob = model.predict_proba(X_proc)
            if y_prob.shape[1] != 2:
                print(f"  Warning: predict_proba output shape is
{y_prob.shape}. Adjusting if possible.")
                if y_prob.shape[1] == 1: y_prob_pos = y_prob.ravel()
                else: raise ValueError(f"Cannot determine positive
class probability from shape {y_prob.shape}")
            else:
                y_prob_pos = y_prob[:, 1]

        threshold = 0.5
        print(f"  Applying threshold {threshold} to probabilities...")
        y_bin = (y_prob_pos >= threshold).astype(int)

        y_pred = np.where(y_bin == 1, 0, -1)
```

```
        print("  Calculating confusion matrix...")
        cm = confusion_matrix(y_orig, y_pred, labels=[-1, 0])
        tn, fp, fn, tp = cm.ravel()

        print("Confusion matrix (true [-1,0] vs pred [-1,0]):\n", cm)
        total = tp + tn + fp + fn
        accuracy = (tp + tn) / total if total > 0 else 0
        precision_0 = tp / (tp + fp) if (tp + fp) > 0 else 0
        recall_0 = tp / (tp + fn) if (tp + fn) > 0 else 0
        specificity_neg1 = tn / (tn + fp) if (tn + fp) > 0 else 0
        f1_0 = 2 * tp / (2 * tp + fp + fn) if (2 * tp + fp + fn) > 0
else 0

        print(f"Accuracy:      {accuracy:.4f}")
        print(f"Precision(0):  {precision_0:.4f}")
        print(f"Recall(0):     {recall_0:.4f}")
        print(f"Specificity:   {specificity_neg1:.4f}")
        print(f"F1(0):         {f1_0:.4f}")

    except AttributeError as ae: print(f"  ERROR evaluating: Attribute
error - maybe preprocessor not found? {ae}")
    except NotFittedError as nfe: print(f"  ERROR evaluating: Model is
not fitted. {nfe}")
    except Exception as e: print(f"  An unexpected error occurred
during evaluation: {e}")

print("\nCalling evaluate_rnn function (Revised Version)...")
evaluate_rnn(final_keras, X_test, y_test_orig, n_features)
```

```
Using KerasClassifier from scikeras.wrappers
--- Running Final RNN Training/Evaluation (User's Structure - Fixed
Again) ---
Instantiating KerasClassifier...
Preprocessing and reshaping training data...
Calculating class weights for final training...
Calculated Class Weights (from y_train_full): {0: 0.5134763870262106,
1: 19.05096618357488}
Fitting final_keras model for 100 epochs...
Model fitting complete.
Preprocessing and reshaping test data...

Calling evaluate_rnn function (Revised Version)...

=== RNN Evaluation (User's Function - Revised) ===
  Preprocessing/reshaping data inside evaluate_rnn...
  Processed data shape: (733042, 1, 11)
  Generating probabilities using model.predict_proba...
  Applying threshold 0.5 to probabilities...
  Calculating confusion matrix...
Confusion matrix (true [-1,0] vs pred [-1,0]):
```

```
 [[649833  60438]
 [  2538  20233]]
Accuracy:      0.9141
Precision(0):  0.2508
Recall(0):     0.8885
Specificity:   0.9149
F1(0):         0.3912

--- Script execution finished (User's Structure - Revised Eval) ---
```

```
evaluate(final_xgb, X_test, y_test_orig)
```

```
=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[702976   7295]
 [ 17137   5634]]
Accuracy:       0.9666703954207262
Precision(0):   0.4357645602908191
Recall(0):      0.24741996398928462
Specificity:    0.9897292723481601
F1(0):          0.31563025210084034
```

```
evaluate(final_mlp, X_test, y_test_orig)
```

```
=== clf ===
Confusion matrix (true [-1,0] vs pred [-1,0]):
 [[707802   2469]
 [ 18970   3801]]
Accuracy:       0.9707533811159524
Precision(0):   0.6062200956937799
Recall(0):      0.16692284045496464
Specificity:    0.9965238620188632
F1(0):          0.2617678454598671
```